DATA STRUCTURE AND ALGORITHMS

LESSON 03 - POINTER

Conceptual Difficulty ★★☆☆☆

Implementation Difficulty ★★☆☆☆

LECTURER: SEAK LENG

OVERVIEW

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Data structure
6. Sub-programs
7. Recursive
8. *Pointers*
9. Linked Lists
10. Stacks and Queues
11. Sorting algorithms
12. Trees

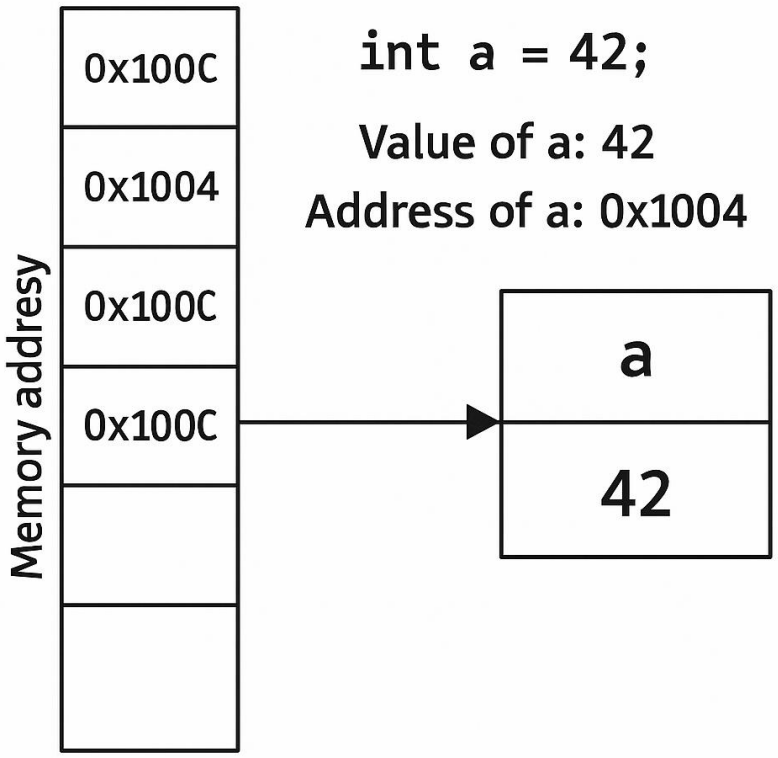
C++

OUTLINE

- What is pointer?
- What are the advantages of using pointer?
- How to use pointer
- Examples and Practice

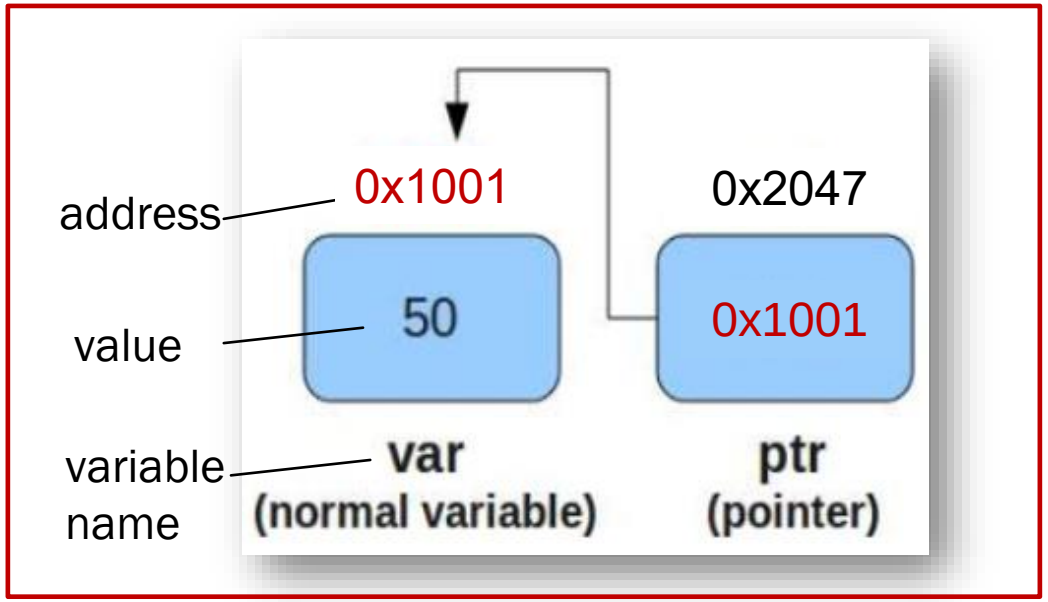
INTRODUCTION

- Every variable in your program is stored at a specified memory location, which has an address.
- A memory address is a unique identifier that tells the computer where data is stored in RAM.
- The address is usually shown as a hexadecimal number, like 0x7ffe5dc.



WHAT IS POINTER?

- A pointer is a variable that holds the memory address of another variable of the same type.
- It points to the location where data is stored in memory rather than storing the data itself.
- Pointers provide direct access to memory. We can access or modify the value at the stored address.



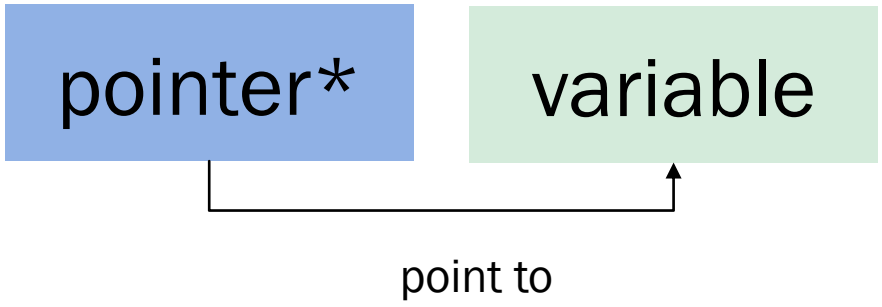
ptr is a pointer to an integer, and it stores the memory address of var.

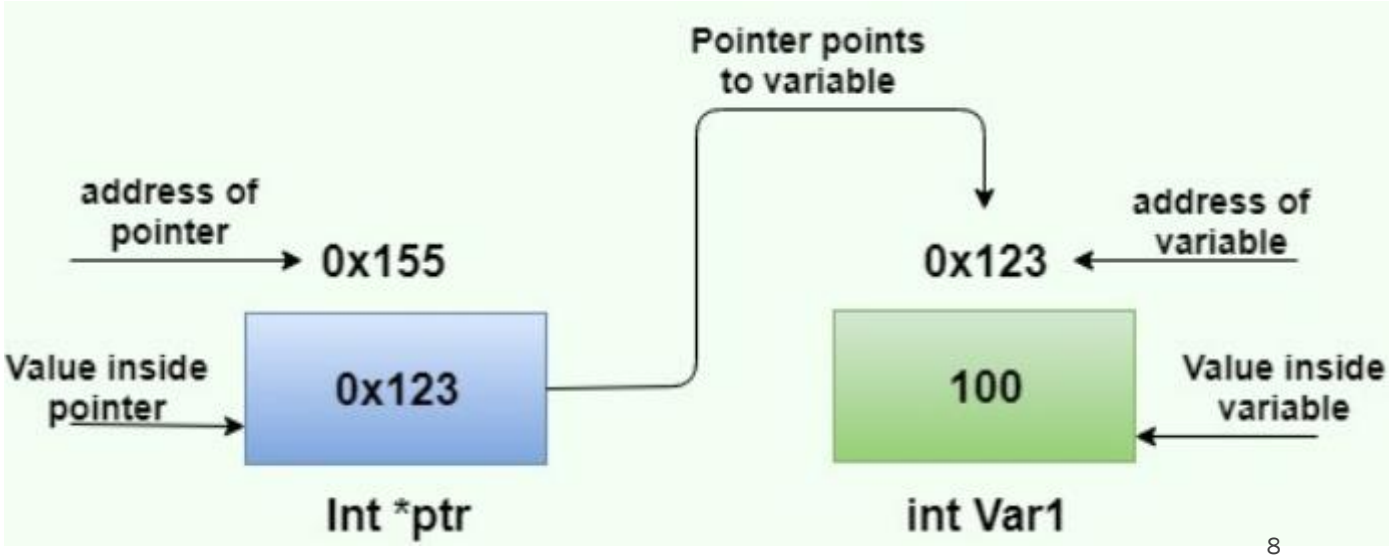
ADVANTAGES OF USING POINTER

1. Use less memory: Dynamic memory allocation
2. Program runs faster, reduce execution time
3. Enable efficient manipulation of data structure: linked list, tree, queue.
4. Provide another way to access array element
5. Instead of copying data, pointer just point to an existing data
6. Enable pass-by-reference in functions
7. Used in low-level system programming

POINTER VS. NORMAL VARIABLE

- A normal variable is used to store value, while *a pointer variable is used to store address (reference) of another variable.*
- Pointers are representation of addresses, the location where a value is stored in memory.





REFERENCE (&) VS. DEREFERENCE (*) OPERATOR

- &: to get memory address of any variable
- *: to get and modify the value stored at the address

Example:

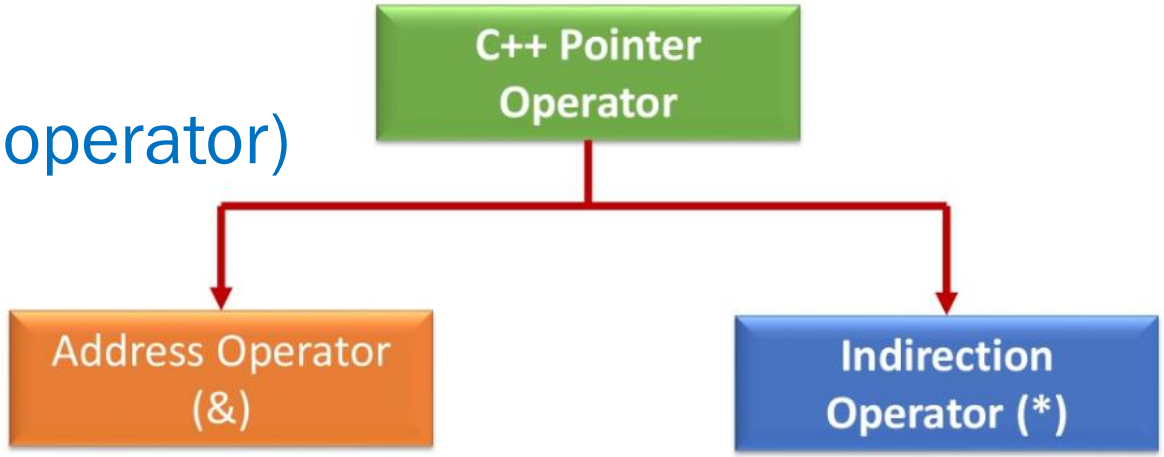
```
int main(){
    int x = 10;
    cout << &x << endl;
    cout << *(&x) << endl;
    *(&x) = 100;
    cout << x << endl;
}
```

Output

```
0x61fe04
10
100
```

POINTER OPERATORS

- Two operators when working with pointer
  - Address operator (**reference** operator)
    - It uses **& (ampersand)**
    - It returns memory address
  - Indirection operator (**dereference** operator)
    - It uses *** (asterisk)**
    - It returns value



POINTER DECLARATION AND INITIALIZATION

- Pointer Declaration: type *nameOfPointer;
- Pointer initialization: type *namOfPointer = &variable;

Example:

```
int a = 5;
int *p = nullptr;
p = &a;
float r = 3.14;
float *pr = &r;
string str = "Sok";
string *ptr = &str;
```

```
cout << p << endl;
cout << pr << endl;
cout << ptr << endl;
```

Output

```
0x61fde8
0x61fde4
0x61fdc0
```

DEREFERENCING A POINTER

- After having declared and initialized a pointer, it is possible to access and modify the contents of the memory address pointed by the pointer using the dereference operator " * ".
- Accessing the value at the address a pointer refers to is called **dereferencing**, done with the * operator.
- Example:

```
int x = 10;
int *p = &x;
int y = *p;
cout << *p << endl;
cout << y << endl;
```

```
*p = *p + 10;
cout << x << endl;
```

Output

```
10
10
20
```

EXAMPLE 1:

```
#include<iostream>
using namespace std;
int main(){
    int num=10;
    int *ptr = &num;
    cout<<"num=\t"<<num<<endl;
    cout<<"&num=\t"<<&num<<endl;
    cout<<"ptr=\t"<<ptr<<endl;
    cout<<"*ptr=\t"<<*ptr<<endl;
}
```

Output

```
num=    10
&num=   0x61fe14
ptr=    0x61fe14
*ptr=   10
```

EXAMPLE 2:

```
#include<iostream>
using namespace std;
int main(){
    int *pc, c;
    c=5;
    cout<<"Address of c: "<<&c<<endl;
    cout<<"Value of c: "<<c<<endl;
    pc = &c;
    cout<<"Address that pc holds: "<<pc<<endl;
    cout<<"Value of address that pc holds: "<<*pc<<endl;
    c = 11;
    cout<<"Address that pc holds: "<<pc<<endl;
    cout<<"Value of address that pc holds: "<<*pc<<endl;
    *pc = 2;
    cout<<"Address of c: "<<&c<<endl;
    cout<<"Value of c: "<<c<<endl;
}
```

Output

```
Address of c: 0x6dfef8
Value of c: 5
Address that pc holds: 0x6dfef8
Value of address that pc holds: 5
Address that pc holds: 0x6dfef8
Value of address that pc holds: 11
Address of c: 0x6dfef8
Value of c: 2
```

PASSING ARGUMENTS TO A FUNCTION

- There are different ways to pass C++ arguments to a function:

Pass by value	Pass by reference (a pointer or reference argument):
- Original variables are unchanged. - Require a full copy of the object - Duplicates the entire object in memory	- Allows functions to modify original variables. - No copy is made (just pass the memory address) - Uses minimal memory (size of a pointer) – memory saving

SEAK LENG

PASSING ARGUMENTS TO A FUNCTION

```
#include<iostream>
using namespace std;
// 1. Pass by value (copy):
void addValue(int a){
    a += 10;
}
// 2. Pass by pointer:
void addValue2(int *a){
    *a += 20;
}
// 3. Pass by reference:
void addValue3(int &a){
    a += 5;
}
```

```
int main(){
    int a = 10;
    addValue(a);
    cout << a << endl;

    int *ptr = &a;
    addValue2(ptr);
    cout << a << endl;

    a = 20;
    addValue3(a);
    cout << a << endl;
}
```

Output

```
10
30
25
```

PASSING ARGUMENTS TO A FUNCTION

- Use *const* references (const int &a) to pass large objects efficiently without modification and prevent accidental modifications.

```
#include <iostream>

using namespace std;

void func2(const int &n){
    //n = 400;
    //error: cannot modify read-only reference
    cout << n/2 <<endl;
    // can read but can't modify n
}
int main(){
    int n = 100;
    func2(n); //no copy is made
    cout << n <<endl; // n remained unchanged
}
```

Output

```
50
100
```

POINTER AND ARRAY

- The name of an array represents the address of its first element &array[0]

```
var A[10], *P: integer
        P ← A
```

- If P points to any element of an array, then P+1 points to the next element
- Pointer Arithmetic:
  - (P+i) points to the ith element behind P and
  - (P-i) points to the ith element before P

POINTER AND ARRAY

- Example: Pointer and Array

```
int arr[5] = {1,2,3,4,5};
int *p = arr; // = &arr[0];
cout<< p << " " << &arr[0] <<endl;
cout<< *p <<" " << arr[0] <<endl;
cout<< *(p+1)<<" "<<arr[1] <<endl;
cout<< *(p+2)<<" "<<arr[2] <<endl;
```

Output

```
0x61fe00 0x61fe00
1 1
2 2
3 3
```

POINTER ARITHMETIC

- Pointers can be incremented/decremented to navigate through memory:

```
int arr[5] = {1,2,3,4,5};
int *p = arr; // = &arr[0];
int n = sizeof(arr)/sizeof(arr[0]);

for(int i=0;i<n;i++){
    cout << p <<": "<< *p <<endl;
    p++;
}
```

Output

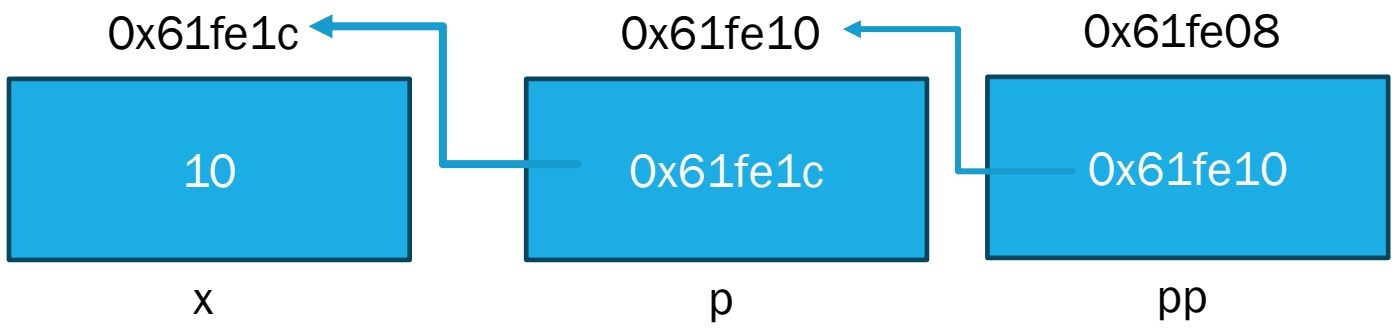
```
0x61fdf0: 1
0x61fdf4: 2
0x61fdf8: 3
0x61fdfc: 4
0x61fe00: 5
```

POINTERS TO POINTERS

- A pointer can point to another pointer variable using (**).

```
int main(){
    int x = 10;
    int *p = &x;
    int **pp = &p;

    cout <<"Address of x: "<< &x <<" stores "<< x <<endl;
    cout <<"Address of p: "<< &p <<" stores "<< p <<endl;
    cout <<"Address of pp: "<< &pp << " stores " << pp <<endl;
    cout <<"*p : " << *p <<endl;
    cout <<"**pp : " << **pp <<endl;
}
```



Output

```
Address of x: 0x61fe1c stores 10
Address of p: 0x61fe10 stores 0x61fe1c
Address of pp: 0x61fe08 stores 0x61fe10
*p : 10
**pp : 10
```

A POINTER TO A STRUCTURE

- The type of variable pointed can be a primary type (such as integer, character) or a complex type (such as structured type)
- We can create a pointer to a structure to access and manipulate its members more efficiently. There are several ways to access members of a structure through a pointer.
  - Dereference and Dot Operator. Ex: (*ptr).member
  - Arrow Operator (->). Ex: ptr->member
  - Array index syntax with pointer. Ex: ptr[0].member;

A POINTER TO A STRUCTURE

```
struct Person {
    int id;
    string name;
};
```

- Example: pointers to a struct

❶
```
Person p1 = {1, "Tom"};
Person *ptr = &p1;
cout << ptr->id <<" "<<ptr->name << endl;
```

❷
```
Person *p2 = new Person;
*p2 = {2, "Jerry"};
cout << (*p2).id << " ";
cout << p2->name << endl;
```

❸
```
Person *p3 = new Person{3, "Jack"};
cout << p3->id << " " << p3->name << endl;
p3->name = "JackyChan";
cout << p3[0].id << " " << p3[0].name << endl;
```

Output

```
1 Tom
2 Jerry
3 Jack
3 JackyChan
```

A POINTER TO A STRUCTURE

- Example: pass a struct pointer to functions

```
#include<iostream>
using namespace std;

struct Employee{
    string name;
    float rate;
};
void printEmployee(const Employee *e){
    cout << e->name << " " << e->rate << endl;
}

void promoteEmployee(Employee *e){
    e->rate += 10;
}
```

```
int main(){
    Employee e={"Jonh",15};

    printEmployee(&e);

    promoteEmployee(&e);

    Employee *ptr = &e;
    printEmployee(ptr);
}
```

Output

```
Jonh 15
Jonh 25
```

A POINTER TO A STRUCTURE ARRAY

```
int main(){
    Person person[3];
    Person *ptr = person;
    for(int i=0;i<3;i++){
        cout <<"Enter id: ";
        cin >> person[i].id;
        cin.ignore(INT_MAX,'\n');
        cout << "Enter name: ";
        getline(cin,person[i].name);
    }
    for(int i=0;i<3;i++){
        cout << (ptr+i)->id <<"\t"<< ptr[i].name<<endl;
    }
}
```

```
struct Person {
    int id;
    string name;
};
```

Output

```
Enter id: 1
Enter name: Tom Jerry
Enter id: 2
Enter name: Lucky Star
Enter id: 3
Enter name: Jacky Chan
1       Tom Jerry
2       Lucky Star
3       Jacky Chan
```

Q&A

PRACTICE

- What is the output of this? 1

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int age;
};

int main() {
    Student *s = new Student{"Alice", 25};
    cout << s->name;
    delete s;
    return 0;
}
```

PRACTICE

- What is the output of this?

2

```
int main(){
    int a = 10, b = 20;
    swap1(a, b);
    cout << a << " " << b<<endl;

    swap2(&a, &b);
    cout << a << " " << b<<endl;

    swap3(a, b);
    cout << a << " " << b<<endl;
}
```

```
void swap1(int a, int b){
    int temp = a;
    a = b;
    b = temp;
}

void swap2(int* a, int* b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

void swap3(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}
```

PRACTICE

- What is the output of this?
- Suppose:

0x61fdf0	0x61fdf4	0x61fdf8
10	20	30
numbers[0]	numbers[1]	numbers[2]

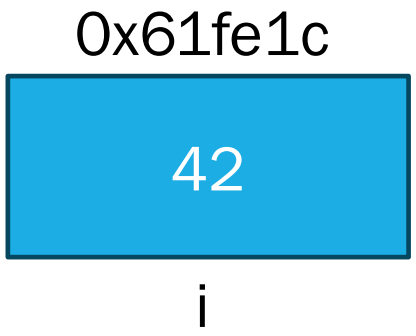
3

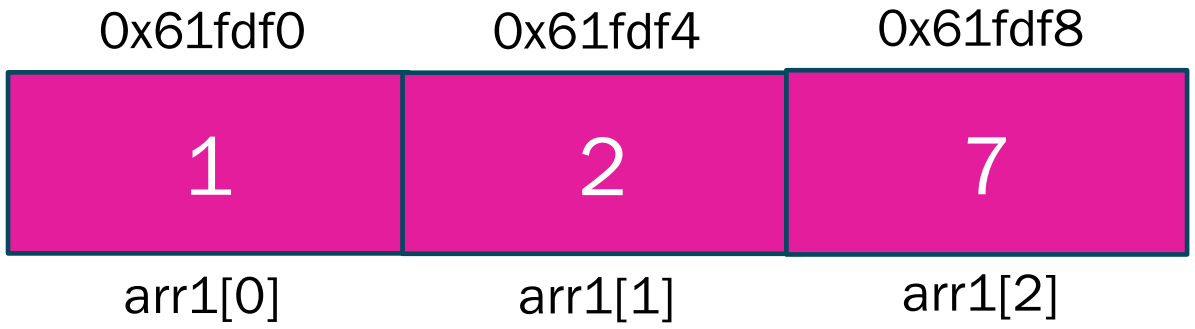
```
#include<iostream>
using namespace std;

int main(){
    int numbers[3] = {10,20,30};
    int *p = numbers;
    cout << *p <<endl;
    cout << *(p+1) <<endl;
    cout << p[2] <<endl;
}
```

PRACTICE

- What is the output of this?
- Suppose:





4

```
#include<iostream>
using namespace std;

void getA(int *pi ){
    cout<<*pi<<endl;
}

int main(){
    int i = 42;
    int *p=&i;
    int arr1[]={1,2,7};
    getA(arr1);
    getA(&i);
    getA(p);
}
```

PRACTICE (USING POINTERS IN C++)

1. Declares and initializes any values to a variable of type float, integer and string. The program then display the value and address (in hexadecimal form) for each variable.
2. Create a subprogram to swap the values of the two integer numbers using pointers.
3. Create an array and use a pointer to traverse and display array elements using pointer arithmetic (not [ ]).

PRACTICE (USING POINTERS IN C++)

3. Find maximum element in array using pointers
4. Reverse an array using pointers.
5. Sum of elements using pointers.
6. Define a struct Student with fields name and score, then use a pointer to access and modify its members.
7. Write a function that takes two integers and returns a pointer to the larger one.

Q&A